

NUCLEO-G474RE

Power Electronics Project Series

Timer Basics

Project #04

Interactive LED Control with Button-Driven Speed Selection

Implementation Frameworks:

STM32CubeIDE | MATLAB Embedded Coder

Document Version: 1.0

Last Updated: April 11, 2026

GitHub: <https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>

This work is licensed under the MIT License

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Applications	3
1.3	Key Concepts	3
1.3.1	Timer Architecture	3
1.3.2	Interrupt Service Routines	3
1.3.3	Event-Driven Architecture	4
1.4	Learning Outcomes	4
1.5	Overall Project Flow	4
2	STM32CubeIDE Implementation	7
2.1	Prerequisites	7
2.2	Hardware Configuration	7
2.2.1	Pin Configuration	7
2.2.2	Peripheral Configuration	7
2.3	Software Architecture	8
2.3.1	Project Structure	8
2.3.2	Initialisation Flow	8
2.3.3	Configuration Images	9
2.4	Code Implementation	10
2.4.1	Main Initialisation	10
2.4.2	Core Functionality	10
2.4.3	Interrupt Service Routines	11
2.4.4	Build and Execution	11
2.5	Debugging	11
3	MATLAB Embedded Coder Implementation	15
3.1	Prerequisites	15
3.2	MATLAB/Simulink Model Design	15
3.2.1	System Components	15
3.2.2	Block Functionality	16
3.3	Model Configuration	17
3.4	Building and Deploying the Simulink Model	17
3.4.1	CubeMX Configuration	17
3.5	Model-to-Code Mapping	18
4	STM32CubeIDE vs. MATLAB Embedded Coder	19
4.1	Implementation Comparison	19
4.2	Key Differences	19
5	Testing and Validation	19
5.1	Performance Characteristics	19

6	Results and Analysis	19
6.1	Expected Outcomes	19
6.2	Key Observations	20
6.3	Project Validation	20
7	Troubleshooting	20
7.1	Common Issues	20
7.2	Debugging	20
8	Extensions and Enhancements	21
8.1	Enhancement 1: Variable Duty Cycle	21
8.2	Enhancement 2: Multi-Speed Motor Control	21
8.3	Enhancement 3: Real-Time Telemetry	21
8.4	Further Learning	21
9	Conclusion	21
9.1	Key Takeaways	21
9.2	Next Steps	21
A	Timer Calculations Reference	23
A.1	Prescaler and ARR Calculation	23
B	Code Listings	23
C	Hardware Documentation	23

1 Project Overview

1.1 Objective

This project creates an interactive LED control system where the on-board green LED (PA5) blinks at user-selectable speeds. By pressing the blue user button (PC13), you cycle through three preset timing intervals: 100ms, 200ms, and 300ms blink periods. The system implements event-driven architecture using timer interrupts and external GPIO interrupts, demonstrating core embedded systems concepts including interrupt service routines (ISRs), timer configuration, and real-time response to external stimuli.

1.2 Applications

Timer-based interrupt systems are fundamental to embedded systems:

- **Real-Time Scheduling:** Periodic tasks like sensor polling, data logging, and control algorithm execution
- **Precision Timing:** Waveform generation, pulse measurement, and time-critical operations
- **Event Detection:** Responding to asynchronous events (button presses, sensor triggers) without blocking main execution
- **Power Electronics:** Gate drive signal generation, current/voltage sampling at precise intervals, closed-loop control

1.3 Key Concepts

1.3.1 Timer Architecture

A timer is a hardware counter that increments at a known frequency. Key components:

- **Prescaler (PSC):** Divides the system clock to set timer frequency. For $PSC = 1699$:
 $f_{\text{tim}} = \frac{170 \text{ MHz}}{1700} = 100 \text{ kHz}$, yielding 10 μs per tick.
- **Auto-Reload Register (ARR):** Defines the counting range. Timer counts $0 \rightarrow ARR$, then resets. With $ARR = 9999$: $T_{\text{update}} = \frac{10000 \text{ ticks}}{100 \text{ kHz}} = 100 \text{ ms}$
- **Update Event:** Generated when counter reaches ARR and resets. Can trigger interrupts for periodic processing.

1.3.2 Interrupt Service Routines

ISRs are special functions triggered by hardware events, enabling asynchronous response without polling. Key characteristics:

- **Low Latency:** Execute within microseconds of event occurrence

- **Automatic Context Save:** CPU automatically saves registers and returns to interrupted code
- **Priority Levels:** Multiple ISRs can be prioritised; higher-priority interrupts preempt lower ones
- **Atomic Execution:** Code within ISR completes without interruption (usually)

1.3.3 Event-Driven Architecture

Rather than polling (continuously checking) for events, the system "wakes up" only when interrupts occur. This conserves CPU cycles and enables deterministic response times.

1.4 Learning Outcomes

By completing this project, you will understand:

- How timer peripherals generate periodic events and interrupts
- Configuring timer prescaler and period (ARR) for desired update frequency
- Implementing ISRs and callback functions to respond to events
- Synchronising asynchronous events (button press) with periodic tasks (LED toggle)
- Data integrity when ISRs modify shared variables
- Transitioning from basic polling-based code to event-driven architecture

1.5 Overall Project Flow

Figure 1 illustrates the complete system operation:

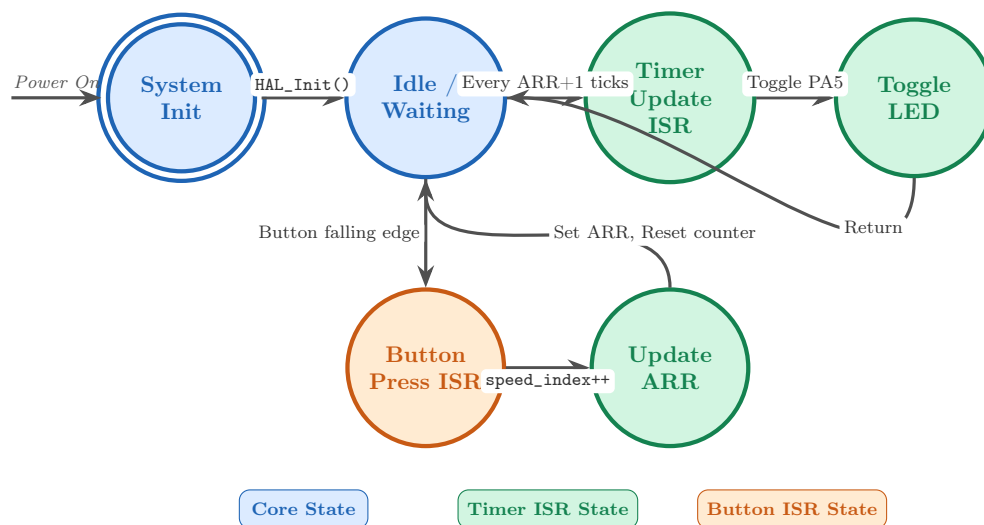


Figure 1: System state machine: initialisation, idle waiting, timer update and button press events.

The system initialises all peripherals, then enters an idle state. The main loop does nothing—all work occurs in ISRs. Timer interrupts toggle the LED every 100/200/300ms. Button interrupts step through speed levels and instantly update the timer period.

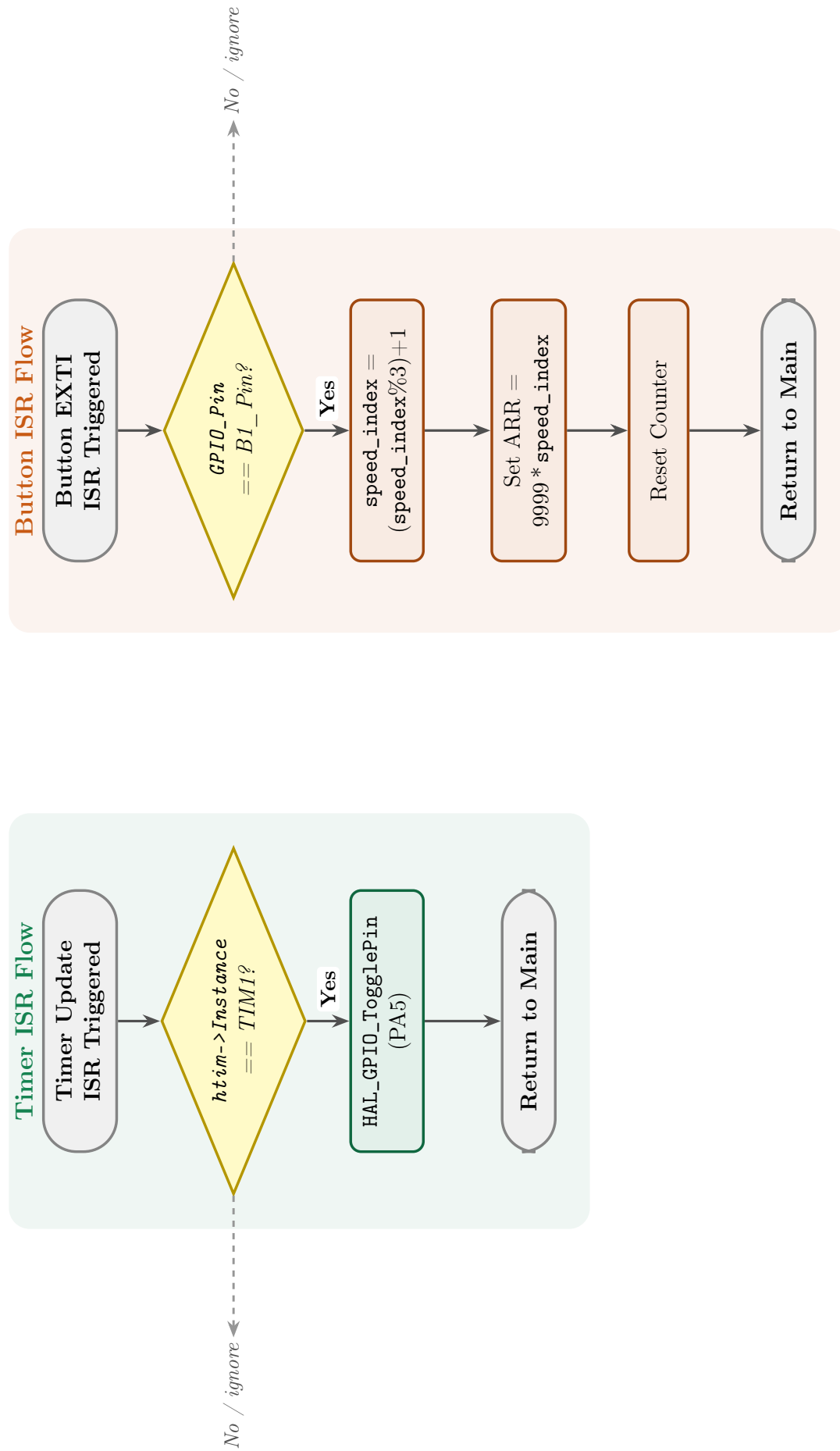


Figure 2: ISR execution flow: Timer ISR toggles LED, Button ISR updates speed and resets counter.

Two ISRs operate independently and asynchronously. Timer interrupts occur every 100/200/300ms; button interrupts occur only on press. Each executes atomically without interference, achieving data integrity through careful design.

2 STM32CubeIDE Implementation

2.1 Prerequisites

Before starting this implementation, ensure you have gone through the previous projects which can be found at <https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>:

- *STM32CubeIDE version 1.10 or later installed*
- *Completion of Projects 01, 02, and 03 for foundational knowledge*
- *NUCLEO-G474RE development board*
- *USB cable for programming*
- *Understanding of GPIO and basic timer concepts*

2.2 Hardware Configuration

2.2.1 Pin Configuration

The project uses two GPIO pins: output for LED control and input for button detection.

Table 1: GPIO Pin Configuration

Pin	Function	Configuration
PA5	LED Output (LD2)	Output Push-Pull, Low Speed, Initially Low
PC13	Button Input (B1)	Input, Falling Edge Trigger, No Pull

2.2.2 Peripheral Configuration

The timer peripheral is configured to generate interrupts at regular intervals:

Table 2: TIM1 Configuration for 100ms Base Period

Parameter	Value and Purpose
Prescaler (PSC)	$1699 \rightarrow \text{Timer clock} = 170\text{MHz} / 1700 = 100 \text{ kHz} (10 \mu\text{s}/\text{tick})$
Counter Mode	Up (counts $0 \rightarrow \text{ARR}$, then resets)
Auto-Reload (ARR)	$9999 \rightarrow \text{Period} = 10000 \text{ ticks} / 100 \text{ kHz} = 100 \text{ ms}$
Auto-Reload Preload	Enabled for synchronisation
Update Interrupt	Enabled in NVIC to trigger ISR on overflow

Note: These configurations are automatically applied when the STM32CubeMX project is created for the NUCLEO-G474RE board. The .ioc (I/O Configuration) file contains all peripheral settings and can be modified through the graphical interface.

Timer formulas used:

$$f_{\text{tim}} = \frac{f_{\text{clock}}}{(\text{PSC} + 1)} = \frac{170 \text{ MHz}}{1700} = 100 \text{ kHz} \quad (1)$$

$$T_{\text{update}} = \frac{(\text{ARR} + 1)}{f_{\text{tim}}} = \frac{10000}{100 \text{ kHz}} = 0.1 \text{ s} = 100 \text{ ms} \quad (2)$$

To change blink speed: $\text{ARR} = 9999 \times \text{speed_index}$ where $\text{speed_index} \in \{1, 2, 3\}$.

2.3 Software Architecture

2.3.1 Project Structure

```

1 04_Timer_Basics/
2 |-- Core/
3 |   |-- Inc/
4 |   |   |-- main.h
5 |   |   '-- stm32g4xx_it.h
6 |   '-- Src/
7 |       |-- main.c
8 |       '-- stm32g4xx_it.c
9 |-- Drivers/
10 |   '-- STM32G4xx_HAL_Driver/
11 '-- .mxproject

```

Listing 1: STM32CubeIDE Project Directory

The `main.c` file contains the initialisation code and the main infinite loop (which does nothing). Two callback functions handle all work: timer ISR and button ISR. The `stm32g4xx_it.c` file is auto-generated and contains the low-level interrupt handlers; no manual modifications are made to this file.

2.3.2 Initialisation Flow

Initialisation follows standard HAL startup:

1. **System Init:** `HAL_Init()` initialises HAL library
2. **Clock Config:** `SystemClock_Config()` sets system clock to 170 MHz
3. **GPIO Init:** `MX_GPIO_Init()` configures PA5 (output) and PC13 (EXTI input)
4. **Timer Init:** `MX_TIM1_Init()` configures prescaler, ARR, and interrupt
5. **Timer Start:** `HAL_TIM_Base_Start_IT(&htim1)` enables timer interrupt
6. **Idle Loop:** Main loop does nothing; all work occurs in ISRs

2.3.3 Configuration Images

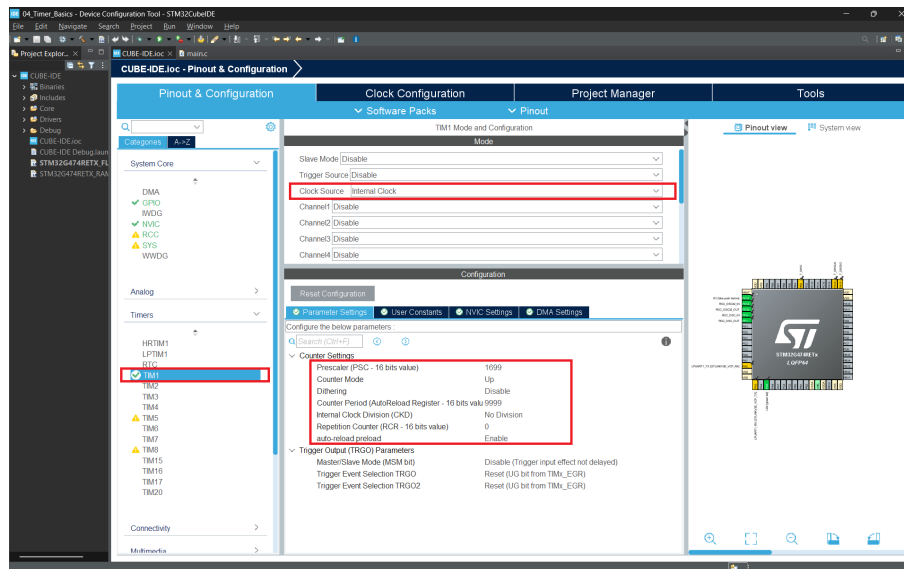


Figure 3: TIM1 prescaler and period configuration. $PSC = 1699$ yields 100 kHz timer clock. $ARR = 9999$ yields 100 ms update period.

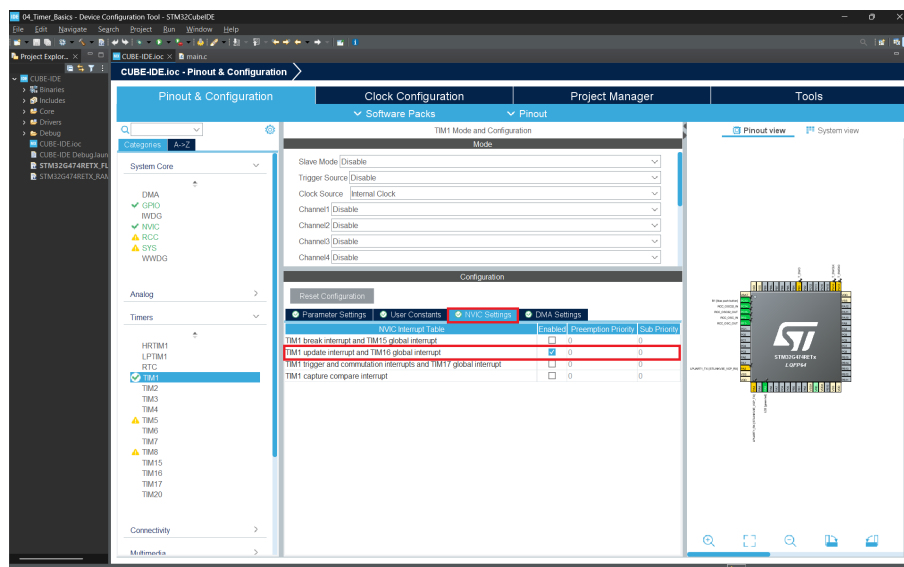


Figure 4: NVIC configuration enabling TIM1 update interrupt. This allows the CPU to respond to timer events.

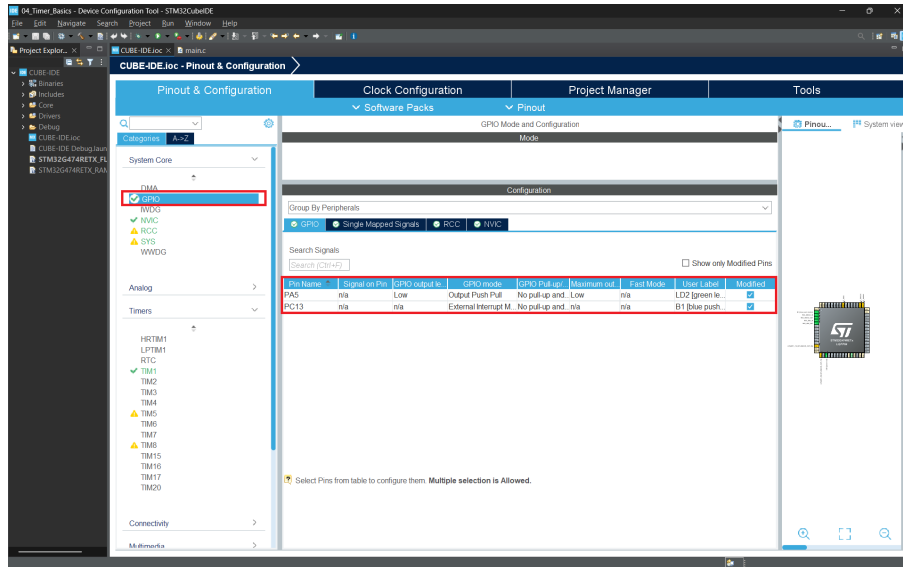


Figure 5: GPIO configuration verified at default state. PA5 configured as digital output, PC13 configured as EXTI falling-edge input. These defaults are correct for LED and button functionality.

2.4 Code Implementation

2.4.1 Main Initialisation

```

1  /* Speed index: 1 -> 100ms, 2 -> 200ms, 3 -> 300ms */
2  static volatile uint8_t speed_index = 1;
3
4  int main(void)
5  {
6      HAL_Init();
7      SystemClock_Config();
8      MX_GPIO_Init();
9      MX_TIM1_Init();
10
11     /* Start timer with interrupt enabled */
12     HAL_TIM_Base_Start_IT(&htim1);
13
14     /* Main loop does nothing; ISRs handle all work */
15     while (1) { }
16 }

```

Listing 2: main.c: Variable Declaration and Initialisation

The `speed_index` variable is declared *volatile* to prevent compiler optimisation, ensuring every read reflects the actual value (important when modified by ISR).

2.4.2 Core Functionality

```

1  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2  {

```

```
3  if (htim->Instance == TIM1) {
4      /* Toggle LED every 100/200/300ms */
5      HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
6  }
7  }
```

Listing 3: main.c: Timer Update Callback

This callback executes every time TIM1 reaches ARR and resets. The LED toggles, creating a square wave.

2.4.3 Interrupt Service Routines

```
1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3     if (GPIO_Pin == B1_Pin) {
4         /* Cycle speed: 1 -> 2 -> 3 -> 1 */
5         speed_index = (speed_index % 3) + 1;
6
7         /* Update timer period instantly (preload disabled) */
8         __HAL_TIM_SET_AUTORELOAD(&htim1, 9999 * speed_index);
9
10        /* Reset counter for immediate effect */
11        __HAL_TIM_SET_COUNTER(&htim1, 0);
12    }
13 }
```

Listing 4: main.c: Button Press Handler

Button press triggers EXTI interrupt. This callback cycles the speed and instantly updates the timer period. Resetting the counter ensures the new period takes effect immediately rather than waiting for the current period to complete.

The `stm32g4xx_it.c` file contains the low-level interrupt handlers (`TIM1_UP_TIM16_IRQHandler`, `EXTI15_10_IRQHandler`) that call `HAL_TIM_IRQHandler` and `HAL_GPIO_EXTI_IRQHandler`, which in turn call the callbacks above. No manual modifications are made to this auto-generated file.

2.4.4 Build and Execution

Build and flash follow standard procedures from previous projects. Connect the board, build the project, and run. The LED blinks at 100ms intervals by default. Press the button to cycle through 100ms → 200ms → 300ms → 100ms.

2.5 Debugging

Observing ISR behaviour requires understanding where interrupt handlers reside and how to inspect them:

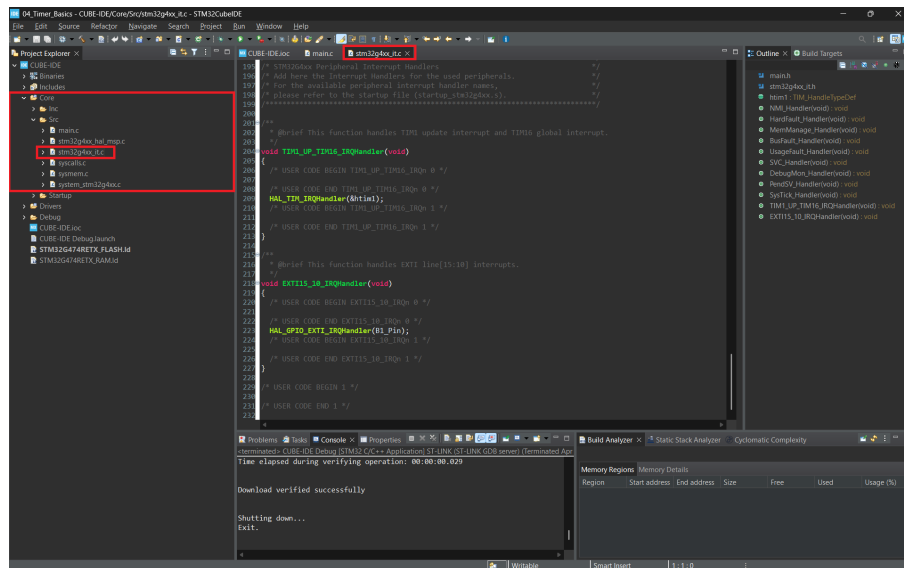


Figure 6: Interrupt handlers are auto-generated in stm32g4xx_it.c. This file contains all low-level ISR implementations.

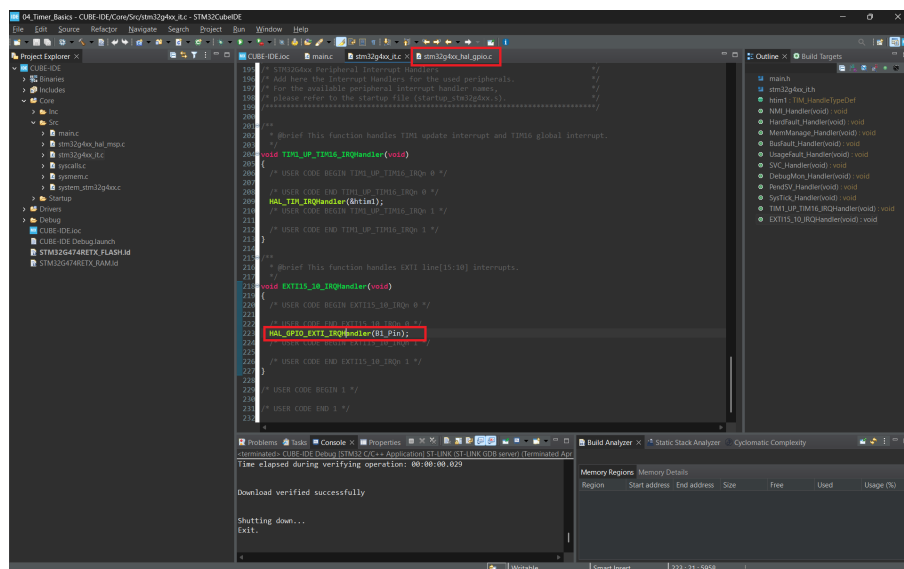


Figure 7: Scroll down in stm32g4xx_it.c to find the specific handler. For button: EXTI15_10_IRQHandler. For timer: TIM1_UP_TIM16_IRQHandler.

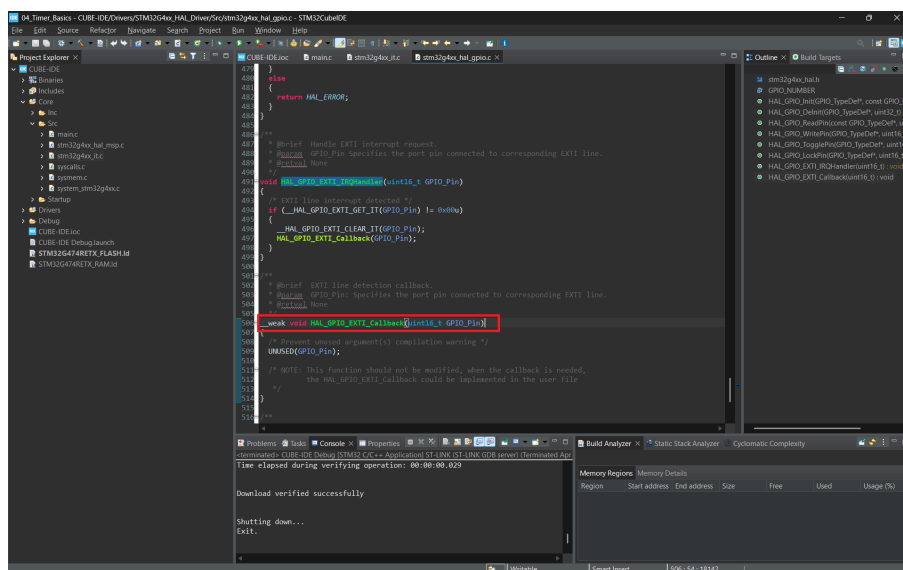


Figure 8: Ctrl+Right-click on HAL_GPIO_EXTI_IRQHandler to navigate to all GPIO interrupt handlers and their callbacks.

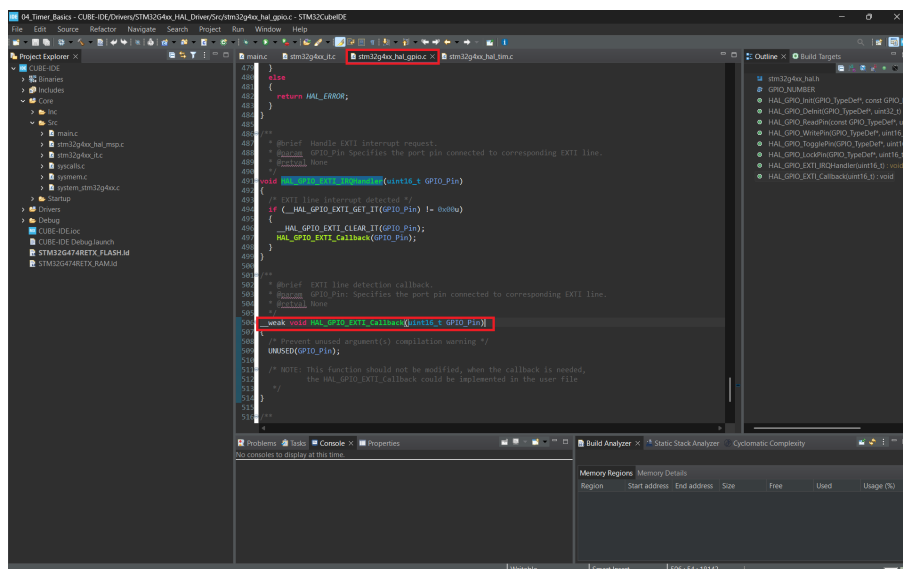


Figure 9: GPIO interrupt callback function. Copy the function name (HAL_GPIO_EXTI_Callback) to override the weak definition in main.c.

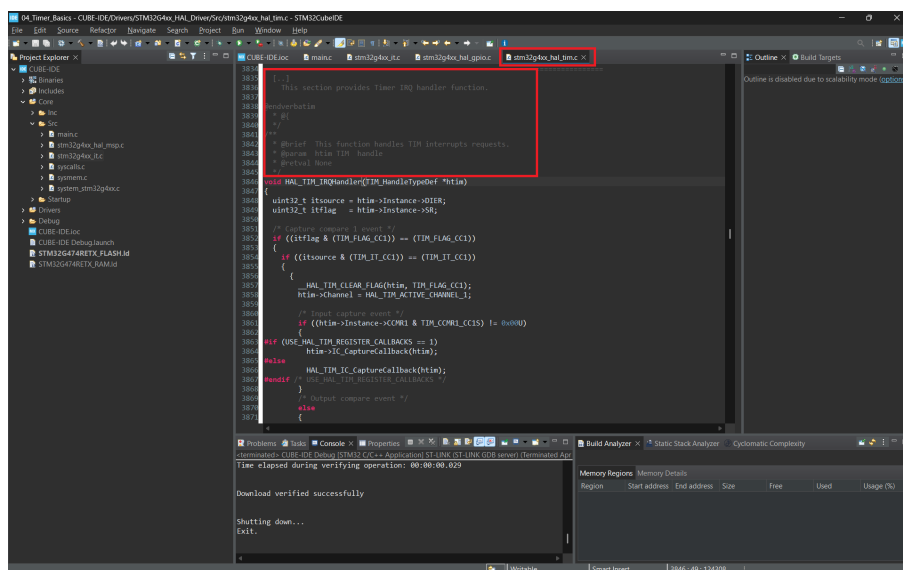


Figure 10: Navigation to timer interrupt handlers showing multiple callback options based on enabled interrupt sources.

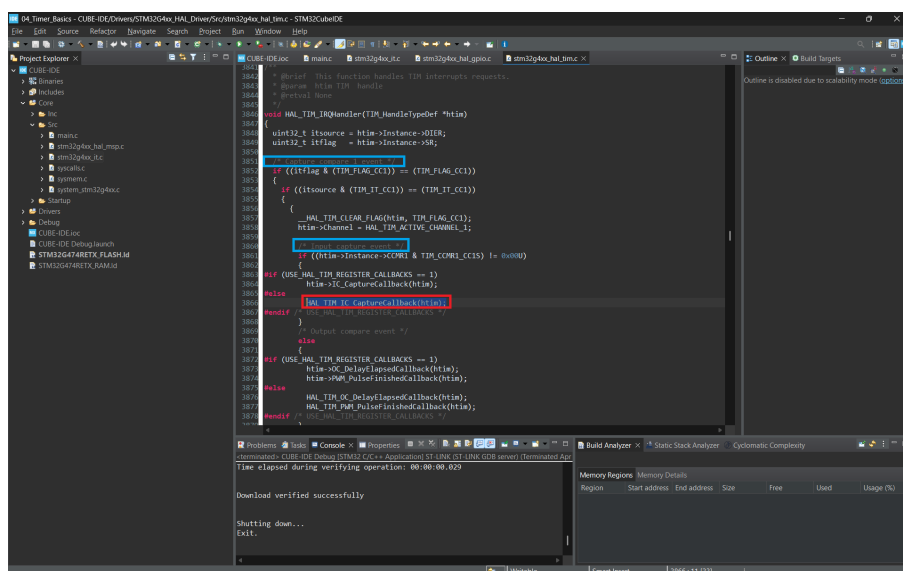


Figure 11: Timer interrupt callbacks. Blue boxes show interrupt types (capture/compare, overflow, etc.). Red boxes show the callback functions to override. For this project, use HAL_TIM_PeriodElapsedCallback (Update event).

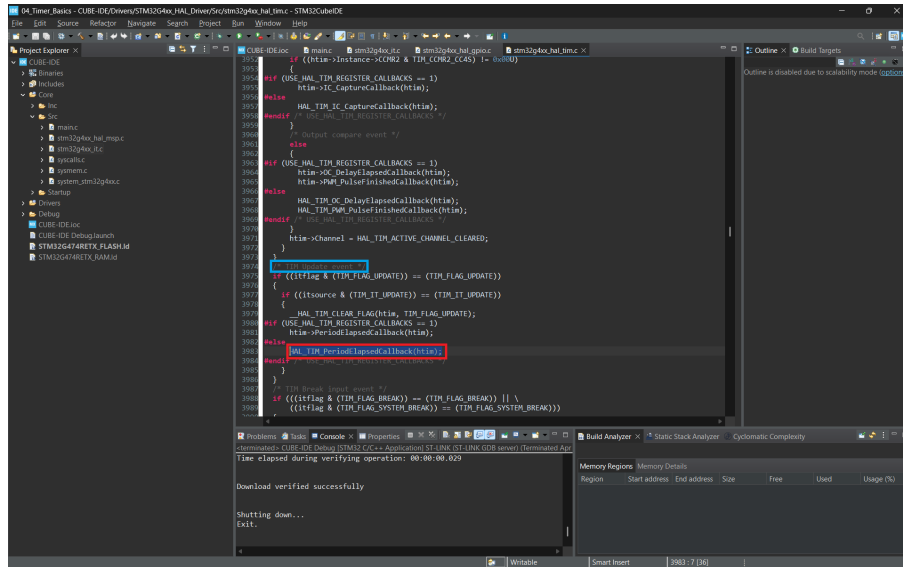


Figure 12: Update interrupt detailed view. The Update event fires when the timer counter reaches ARR and resets. This triggers our LED toggle callback.

3 MATLAB Embedded Coder Implementation

3.1 Prerequisites

Before starting the MATLAB Embedded Coder implementation, ensure you have gone through the previous projects available at NUCLEO-G474RE-PowerElectronics-Guide:

- *MATLAB R2021b or later with Embedded Coder toolbox*
- *Embedded Coder Support Package for STMicroelectronics STM32*
- *ARM GCC Compiler set to **GNU Tools for ARM Embedded Processors***
- *NUCLEO-G474RE development board*
- *Completion of Projects 01-03*

Note: The ARM GCC compiler configuration is critical. This must be changed to **GNU Tools for ARM Embedded Processors** in MATLAB preferences for successful code generation.

3.2 MATLAB/Simulink Model Design

The Simulink model implements the same functionality as the CubeIDE version using event-driven blocks:

3.2.1 System Components

The model consists of three main subsystems:

1. **Button Input Subsystem:** Reads PC13 and generates speed_index (1 → 2 → 3 → 1)
2. **LED Toggle Subsystem:** Toggles output to PA5 when timer interrupt occurs
3. **Timer Interrupt Block:** Triggers at configured intervals

Critical design: ISR code is placed in triggered subsystems. These execute only when an interrupt fires, simulating ISR behaviour. Rate transition blocks manage data flow between different execution domains (timer interrupt vs. button interrupt).

3.2.2 Block Functionality

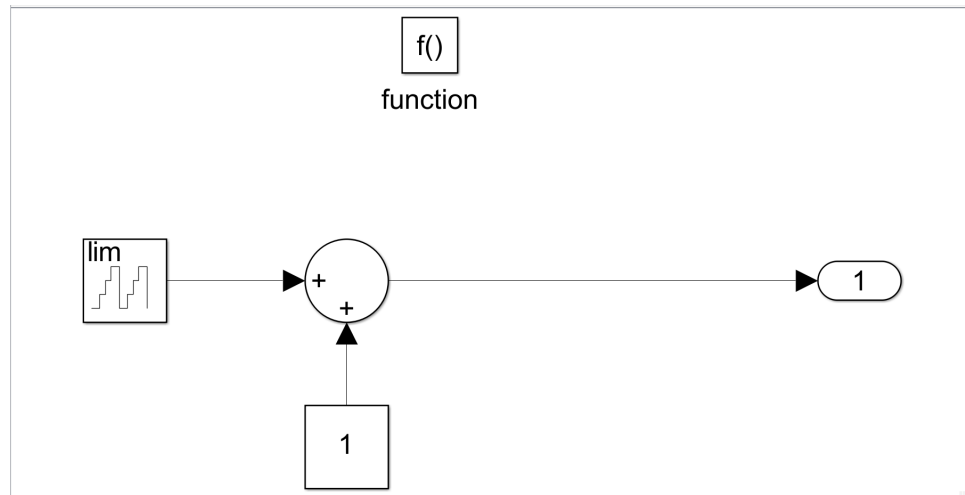


Figure 13: Button subsystem: Triggered on EXTI13 falling edge. Counter counts $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$, multiplied by 9999 to generate ARR values (9999, 19998, 29997).

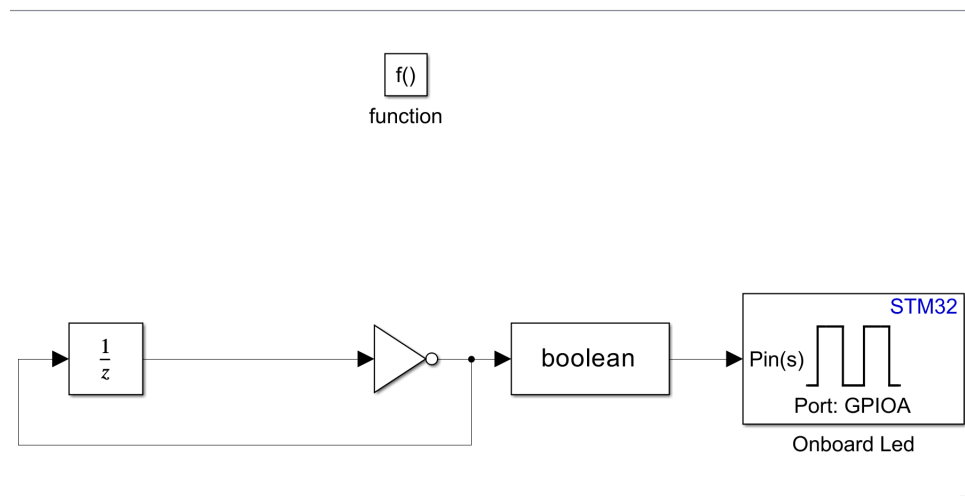


Figure 14: LED subsystem: Triggered on timer update interrupt. Unit delay creates feedback loop; NOT gate inverts. Output drives PA5 and feeds back, toggling state on each interrupt.

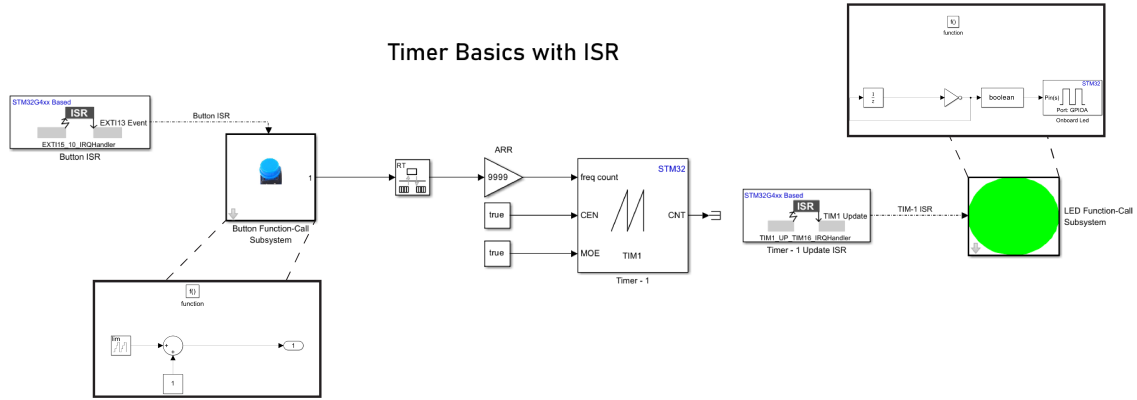


Figure 15: Complete Simulink model showing timer interrupt source, button interrupt source, LED toggle subsystem, and button counter subsystem. Scopes monitor outputs.

3.3 Model Configuration

3.4 Building and Deploying the Simulink Model

The Embedded Coder Support Package for STM32 simplifies the build and deploy process by automating code generation and compilation.

3.4.1 CubeMX Configuration

After hardware selection, the .ioc file is created in STM32CubeMX. Verify the following configurations:

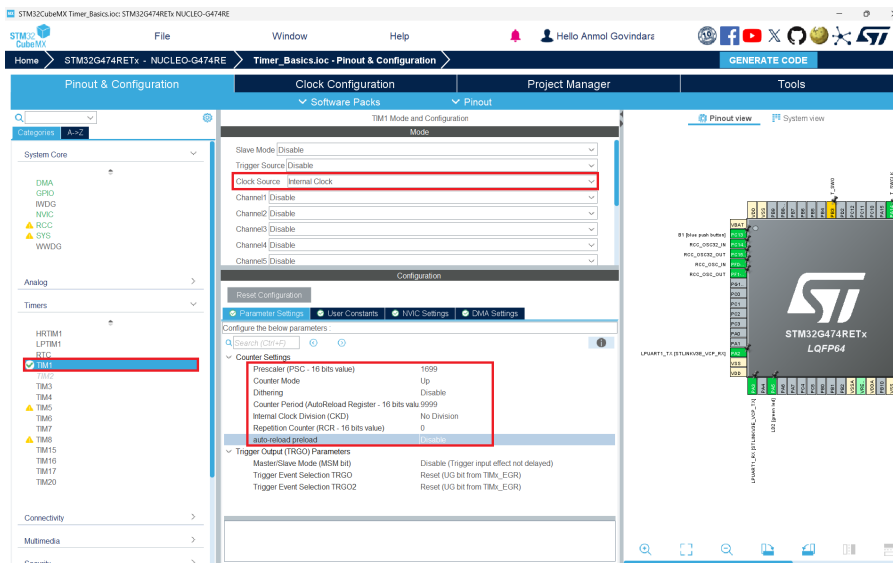
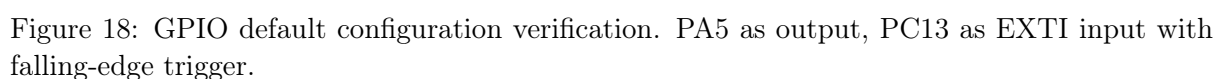
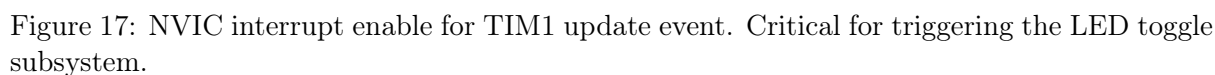


Figure 16: TIM1 prescaler and period configuration in CubeMX (accessed through Simulink). PSC = 1699, ARR = 9999.



Simulink generates C code from blocks. The triggered subsystems map directly to ISR callbacks:

- **Button Subsystem** (triggered by EXTI13) → Generated as function called from HAL_GPIO_EXTI_Callback
- **LED Subsystem** (triggered by TIM1 update) → Generated as function called from HAL_TIM_PeriodElapsedCallback
- **Rate Transition Blocks** → Protect data integrity when ISRs access shared variables

The generated code maintains the same logic as hand-written CubeIDE code but with automatic error handling and optimisation.

4 STM32CubeIDE vs. MATLAB Embedded Coder

4.1 Implementation Comparison

Table 3: Implementation Comparison

Aspect	STM32CubeIDE	MATLAB Embedded Coder
Code Style	Manual C with HAL function calls	Auto-generated from Simulink blocks
ISR Implementation	Callbacks in main.c, handlers in stm32g4xx_it.c	Triggered subsystems in model
Debugging	Step through C code, inspect variables	Simulate model before deployment
Data Integrity	Manual volatile declarations	Automatic rate transitions
Development Time	Longer, requires detailed knowledge	Faster, visual design

4.2 Key Differences

STM32CubeIDE Implementation: Direct control over ISR execution. Developer explicitly declares *volatile* variables, manages data races, and uses atomic operations. Code is compact and efficient but requires deep understanding of microcontroller internals.

MATLAB Embedded Coder Implementation: Visual representation of system architecture. Triggered subsystems simulate ISR behaviour. Rate transitions automatically protect data. Generated code may be larger but more robust against subtle synchronisation bugs.

5 Testing and Validation

5.1 Performance Characteristics

- **Blink Frequencies:** 10 Hz (100ms), 5 Hz (200ms), 3.33 Hz (300ms) corresponding to speed_index 1, 2, 3
- **Button Response Latency:** < 1 ms; new blink rate takes effect within one timer period
- **CPU Utilisation:** < 1% in idle; main loop consumes negligible power

6 Results and Analysis

6.1 Expected Outcomes

STM32CubeIDE Implementation: LED blinks at 100ms intervals by default. Pressing button steps through 200ms, 300ms, and back to 100ms. Transitions are instantaneous due to counter reset.

MATLAB Embedded Coder Implementation: Identical observable behaviour. Generated code produces equivalent timing and response.

6.2 Key Observations

1. ISR execution is non-blocking; button presses do not interrupt timer ISR and vice versa
2. Instant ARR update and counter reset ensure smooth transitions without glitches
3. Event-driven architecture consumes minimal power compared to polling
4. Data integrity maintained through atomic operations and proper volatile declarations

6.3 Project Validation

This project successfully demonstrates:

- Event-driven architecture with multiple interrupt sources
- Data synchronisation between asynchronous ISRs
- Timer period calculation and ARR configuration
- Real-time response to external stimuli (button press)

7 Troubleshooting

7.1 Common Issues

Table 4: Troubleshooting Guide

Issue	Solution
LED does not blink	Verify PA5 configuration and <code>HAL_TIM_Base_Start_IT()</code> call
Button does not change speed	Check PC13 EXTI configuration and falling-edge trigger setting
Speed change is delayed	Verify counter reset (<code>__HAL_TIM_SET_COUNTER</code>) in button ISR
MATLAB build fails	Ensure ARM GCC compiler set to GNU Tools for ARM Embedded Processors

7.2 Debugging

Use the integrated STM32CubeIDE debugger to set breakpoints in callbacks and inspect variables. Monitor the `speed_index` variable to verify button presses increment it correctly. Use an oscilloscope on PA5 to confirm blink frequency matches ARR configuration.

8 Extensions and Enhancements

8.1 Enhancement 1: Variable Duty Cycle

Extend the project to control not just frequency but also duty cycle. Add a second button to adjust the LED on-time independently.

8.2 Enhancement 2: Multi-Speed Motor Control

Use the same timer interrupt mechanism to generate PWM signals for motor speed control, extending into power electronics applications.

8.3 Enhancement 3: Real-Time Telemetry

Add UART communication to transmit button presses and timer events to a computer for logging and analysis.

8.4 Further Learning

Advanced timer topics:

- PWM generation with precise phase and frequency control
- Capture-compare for measuring input signal frequency and duty cycle
- Timer synchronisation and cascading for complex waveforms
- DMA-driven periodic operations without ISR overhead

9 Conclusion

This project introduced event-driven architecture through practical implementation of timer and button interrupts. Rather than polling, the system responds immediately to external stimuli, demonstrating core embedded systems design patterns used in power electronics and real-time control.

9.1 Key Takeaways

- Timers generate periodic events for precise, real-time operations
- ISRs enable asynchronous event response without blocking main execution
- Careful variable management (volatile, atomic operations) ensures data integrity when multiple ISRs access shared data
- Event-driven design is more efficient and responsive than polling-based approaches

9.2 Next Steps

Subsequent projects build on interrupt and timer concepts:

References

References

- [1] STMicroelectronics, *STM32G4 Series Advanced ARM-based 32-bit MCUs Reference Manual*, RM0440.
- [2] STMicroelectronics, *STM32G474RE Datasheet*.
- [3] STMicroelectronics, *STM32G4 NUCLEO-64 Boards User Manual*, UM2505.
- [4] STMicroelectronics, *STM32CubeIDE Integrated Development Environment User Manual*, DM00629855.
- [5] STMicroelectronics, *Description of STM32G4 HAL and Low-Layer Drivers*, UM2570.
- [6] MathWorks, *Embedded Coder Documentation*.
- [7] MathWorks, *Embedded Coder Support Package for STMicroelectronics STM32*.

A Timer Calculations Reference

A.1 Prescaler and ARR Calculation

For desired blink frequency f , period $T = 1/f$:

$$\text{ARR} = \frac{f_{\text{tim}} \times T}{(\text{Prescaler} + 1)} - 1 \quad (3)$$

For our system: $f_{\text{tim}} = 100 \text{ kHz}$, Prescaler = 1699

- 100 ms period: $\text{ARR} = (100,000 \text{ Hz} \times 0.1 \text{ s}) - 1 = 9,999$
- 200 ms period: $\text{ARR} = (100,000 \text{ Hz} \times 0.2 \text{ s}) - 1 = 19,999$
- 300 ms period: $\text{ARR} = (100,000 \text{ Hz} \times 0.3 \text{ s}) - 1 = 29,999$

B Code Listings

Complete main.c and stm32g4xx_it.c source code available in the GitHub repository: https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/tree/main/Examples/04_Timer_Basics

C Hardware Documentation

Refer to the official NUCLEO-G474RE schematic and STM32G474RE datasheet for complete pin definitions and timer capabilities.

Author

Anmol Govindarajapuram Krishnan

GitHub: <https://github.com/Anmol-G-K>

Email: cb.en.u4eee23103@cb.students.amrita.edu

Acknowledgments

We acknowledge the following for their contributions and support:

- **STMicroelectronics** for providing comprehensive microcontroller documentation and development tools
- **MathWorks** for excellent MATLAB and Embedded Coder resources
- The embedded systems and power electronics community for innovation and knowledge sharing
- Contributors and reviewers providing valuable feedback

This project is licensed under the MIT License.

For updates and additional resources:

<https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>